Performance Evaluation Methods – Notes

Francesco Passino

Contents

0.1 How to build a good simulator – lessons 29/09/2026 . . . 2
0.1.1 Events . . . 2
0.1.2 min-heap trees . . . 2
0.1.3 Statistical counters . . . 2
0.1.4 Functions . . . 3

1 Lesson 02 — 14/10/2026 **3**
1.1 Giving inputs to the simulator . . . 3
1.1.1 Trace-driven simulation . . . 3
1.1.2 Self-driven simulation . . . 3
1.1.3 Mixed approach . . . 4
1.2 Generating random values from a distribution . . . 4
1.2.1 Random number generation . . . 4
1.2.2 Linear Congruential Generators (LCGs) . . . 5
1.2.3 Independent streams of random numbers . . . 6
1.3 Testing RNGs . . . 6
1.4 Random variate generation . . . 7
1.4.1 Inverse transform . . . 7

0.1 How to build a good simulator – lessons 29/09/2026

to build a good simulator, we need to define some essential components.

0.1.1 Events

An event is represented by a data structure including a **firing time** field. We also include a **priority** field, which allow us to choose the event to schedule if two events append at the same time. When we have a set of events, we need another data structure to store them. Usually we use an event queue, possibly sorted by firing time. The minimum operations on the event queue are:

- **Extraction** of the nearest future event.
- **Insertion** in order of a new event into the queue.
- **Deletion** of an event from the queue.

A simple implementation of an event queue is a linked list, where the events are sorted by firing time. For the sorted list we have a complexity of $O(n)$ for insertion and deletion, and $O(1)$ for extraction. We will see other possible better implementations of the event queue in the next lessons. In general handling the event queue is a big part of the total complexity of the simulator. To have fast and scalable simulators, we need to have a fast event queue implementation. Some commonly used data structures are *min-heap* trees and *calendar queues*.

0.1.2 min-heap trees

They are binary trees with the following properties:

- The value of each node is less than or equal to the values of its children.
- The tree is complete, i.e., all levels are completely filled except possibly for the last level, which is filled from left to right. So, depth $O(\log n)$.

A min-heap tree can be implemented using an array, where the root is at index 0, and for each node at index i, its left child is at index $2i + 1$ and its right child is at index $2i + 2$. The parent of a node at index i can be found at index $\lfloor (i - 1)/2 \rfloor$. Implementing a min-heap tree using an array allows for efficient advantages in terms of memory usage and cache performance.

Implementing the event queue using a min-heap tree allows for efficient insertion and extraction of events. This is because the first element of the array is the closest in the future so for extraction we just extract from the head. After the extraction, we need to restore the heap property by performing the following steps:

1. Replace the root of the heap with the last element in the array.
2. swap the root with its smallest child until the heap property is restored.

The complexity of this operation is $O(\log n)$.
If we need to insert a new event, we can add it to the end of the array and then swap it with its parent until the heap property is restored. The complexity of this operation is also $O(\log n)$.
If we delete an event from the queue, we first need to find the event in the array, which has a complexity of $O(n)$. Then we need to "reheapify" the tree, which has a complexity of $O(\log n)$. So the total complexity of deletion is $O(n)$. Luckily teh deletion is not that common in simulation.

0.1.3 Statistical counters

statistical counters are used to collect information about the simulation. They can be used to compute averages, variances, and other statistics about the events that occur during the simulation.

0.1.4 Functions

We need a set of functions to realize the simulation. The main functions are:

- **Initialization**: assigning the initial values to the variables and data structures used in the simulation.
- **Event scheduling**: schedule the next event to occur in the simulation.
- **Event handling**: update the state and the statistical variables. Possibly insert new events in the event queue.
- **Statistics computation**: compute the statistics of interest at the end of the simulation.

We have special handlers, such as **init**, which is called at the beginning of the simulation to initialize the system and put the first event in the event queue.

1 Lesson 02 — 14/10/2026

1.1 Giving inputs to the simulator

We can have `trace-driven` simulation, where the input is taken from a trace obtained by measuring the real system. A second approach is `self-driven` simulation, where inputs are artificially generated as random inputs from a probability distribution. To find the correct distribution we can gather data from the real system and perform statistical analysis (goodness-of-fit tests), read existing papers on the subject or use common sense.

1.1.1 Trace-driven simulation

Pros:

- `Real-life case study`.
- Good for *validation* purposes.
- It might be the only option (e.g., compressed video, which is hard to model with synthetic generators).

Cons:

- `Overhead`: we need to obtain the trace.
- `Storage`: the trace file could be big.
- `Lack of generality`: traces are only for one specific case.
- `Inflexibility`: traces cannot be tuned for other problems.

1.1.2 Self-driven simulation

Pros:

- `Compactness`.
- `Known statistical properties`.
- `Tunability`.

Cons:

- Identification of the right distributions may be difficult or even impossible.

If it's impossible to derive a distribution we can use an `empirical PMF (EPMF)` from available samples. With this approach we may have problems in defining ranges and bins (they influence the PMF), and we can have overflowing samples (*censored data*). To take into account the probability of values larger than the range of the EPMF, we can add an exponential tail to the right. We will use this approach only if it's the only option available, because we will have loss of information, we may need a lot of storage to memorize it and tuning is tricky.

1.1.3 Mixed approach

When the simulator has multiple inputs, we can mix the two approaches. For example:

- MPEG video packets (obtained from a real video file) transmitted to nodes that move according to a theoretical distribution.
- Exponentially distributed packets transmitted to nodes that move along predetermined trajectories (based on real system observations).

1.2 Generating random values from a distribution

We need to generate random values if we want to use self-driven simulation. Random values are used for input generation, but also inside the model itself (e.g., to decide whether a transmitted packet is correctly received at the destination). We will follow these two steps:

1. `Random number generation`: generate a random variable uniformly distributed within $[0, 1)$.
2. `Random variate generation`: transform the previously generated variable in order to meet the desired distribution.

1.2.1 Random number generation

`Midsquare method`:

1. Start with a 4-digit positive integer Z_0.
2. Square it to obtain an (up to) 8-digit integer, adding zeros to the left if needed.
3. Take the middle 4 digits and call them Z_1.
4. Place a decimal point at the left of Z_1 to obtain $U_1 \in [0, 1)$.
5. Repeat from step 2 with Z_1 as the starting number.

Example 1.1.
Start with $Z_0 = 7182$ (the first four decimal digits of e):

Z_{i-1}	Z_{i-1}^2	Z_i	U_i
7182	51581124	5811	0.5811
5811	33767721	7677	0.7677
7677	58936329	9363	0.9363
9363	87665769	6657	0.6657

The numbers look random. However, starting from $Z_0 = 1009$ we get 0180, 0324, 1049, 1004, 0080, 0064, 0040, 0016, 0002, 0000, and from there the sequence stays at 0 forever.

This algorithm looks good but could quickly degenerate to 0, and also, is it actually random? The truth is that it is impossible to generate random numbers with a deterministic algorithm. Luckily we don't really need random numbers: we need something that generates numbers in $[0, 1)$ that are uniformly distributed and uncorrelated. Of course, a good generator must be fast and parsimonious with memory usage, and we want it to be `reproducible`. This looks counterintuitive, but it is essential for debugging and fair comparison of alternatives. Finally, the generator should provide `separate streams` of random numbers: we can think of them as separate and independent generators, so that we can dedicate a particular stream to a particular source of randomness in the simulation.

What we want are `pseudo-random` numbers, generated by a deterministic function f of the previous values:

$$x_n = f(x_{n-1}, x_{n-2}, \ldots, x_0),$$

starting from an input called `seed` (x_0; quite often only one seed is needed). If we use the same function and seed we always get the same results. Every such generator has a `period`: sooner or later it produces a value it has already produced, and from that point the sequence starts again (loop). To choose a good function f we want subsequent values to be uncorrelated and uniformly distributed, f to be efficiently computable, and the period to be large.

1.2.2 Linear Congruential Generators (LCGs)

This is an example of a good random number generator. Notice that the formula is not complex (complexity does not imply randomness).

Definition 1.2 (Linear congruential generator)**.**

$$x_n = (a \cdot x_{n-1} + b) \bmod m,$$

where $0 < m$, $a < m$, $b < m$, $x_0 < m$. The numbers that appear to be IID $U(0, 1)$ are $x_0/m, x_1/m, \ldots, x_n/m$.

We can have two cases of LCG depending on the parameter b:

- $b = 0$: `multiplicative LCG`;
- $b > 0$: `mixed LCG`.

Example 1.3.
Consider the LCG with $m = 16$, $a = 5$, $b = 3$ and $x_0 = 7$:

$$x_n = (5x_{n-1} + 3) \bmod 16.$$

It generates $7, 6, 1, 8, 11, 10, 5, 12, 15, 14, 9, 0, 3, 2, 13, 4$, and then $x_{16} = 7$, $x_{17} = 6$, ... From $n = 16$ to 31 we obtain exactly the same values as from $n = 0$ to 15, in the same order. Here $P = 16 = m$, consistent with the conditions below ($m = 2^4$, $a = 4 \cdot 1 + 1$, b odd).

The period P of the LCG is at most m (it generates numbers in $\{0, \ldots, m-1\}$), and it can be smaller depending on the other parameters. If $P = m$ then the LCG is said to have `full period`. With mixed LCG, sufficient conditions to have full period are:

- m is a (large) power of two;
- $a = 4c + 1$, for some integer c;
- b is an odd number.

For multiplicative LCG it is impossible to obtain full period, because 0 is mapped to 0: if the seed is 0 we remain stuck at 0, and a sequence starting from a nonzero seed can never reach 0, so $P \leq m-1$. If we choose m to be a power of two, the period cannot be larger than $m/4$. This max period $m/4$ is obtained with $a = 8c \pm 3$ and an odd initial seed.

For both LCGs we can choose m to be a power of 2 to increase the performance of the generator, since the modulo operation is easy.

If the conditions for full period are met, any initial seed will do; otherwise the period depends on the seed. In general we should avoid even seeds (with multiplicative LCGs), zero, and unpredictable seeds such as the clock value or the process ID, which make the simulation impossible to reproduce.

1.2.3 Independent streams of random numbers

In case of multiple inputs (e.g., packet arrival times and packet sizes) we need to generate several streams of random numbers. If we use the same stream for different inputs, we cannot guarantee that the subsampled streams are uniform and uncorrelated. Moreover, changing one parameter of one input changes the values of all the others, which makes debugging and fair comparisons almost impossible. So we never use the same RNG for more than one thing: we use a replica of the same RNG for each input, and we seed each replica so that its substream starts in a different position of the main stream, far away in the period. If we need n streams with less than k numbers each and the period is $P \geq nk$, we can use the seeds

$$s_i = x_{(i-1)k}, \qquad 1 \leq i \leq n.$$

LCGs also have some known limits: every x_i can be computed deterministically from x_0 (which is useful to find these seeds), the values can only be multiples of $1/m$, and tuples of subsequent values lie on a finite number of hyperplanes, so many sequences will never come out.

We should never use the C/C++ `rand()` function in simulations: its algorithm is compiler-specific (so results are not portable), it is not guaranteed to pass any statistical test and it does not provide independent streams. Other RNG algorithms exist, such as quadratic congruential, multiplicative recursive, Tausworthe and `Mersenne-Twister` generators; the latter has a huge period ($2^{19937}-1$) but needs about 2 KB of memory per instance.

1.3 Testing RNGs

The first thing we need to check is: are the numbers really uniformly distributed within $[0,1)$?

We can answer this question with different tests:

1. `QQ plot`: we run one long sequence and plot its quantiles against the quantiles of $U(0,1)$. We need to obtain a straight line, and we check for deviations, especially at the tails.

2. `Chi-square test` at a level $1-\alpha = 0.9$: we draw a large sample of size n and divide $[0,1)$ into k buckets of equal width, such that $n/k > 10$. Bucket i contains o_i elements, while we expect $e_i = n/k$. We compute

$$D = \sum_{i=1}^{k} \frac{(e_i - o_i)^2}{e_i},$$

   and if $D > \chi^2_{k-1,1-\alpha}$ we reject the $U(0,1)$ hypothesis. It works with large samples (~ 1000), and its result depends a lot on the choice of the buckets.

3. `Kolmogorov-Smirnov test`: like the chi-square test, but we compare the ECDF of the sample with the theoretical CDF. We draw a (not too large) sample, sort it, and compute the maximum vertical distances D^+ and D^- between the ECDF and the CDF of $U(0,1)$. If either is larger than the value of the KS distribution (from tables, it depends on n and on

the confidence level), we reject the $U(0,1)$ hypothesis. It works better than the chi-square test with small samples.

The second property we need to check is: are the values independent?

We can check correlation numerically with a `correlogram` (we will see it at the end of the course), or use visual inspection with `lag plots`, i.e., we plot the pairs (x_i, x_{i+k}) for various lags k. If the graphs show any trend, the numbers are correlated.

Many other tests are possible. Some are `experimental tests`, which test the generated sequence (e.g., the chi-square test in k dimensions, $k = 2, 3, \dots$), others are `theoretical tests`, which test the algorithm itself (e.g., the *spectral test*). The underlying message is: never use an untested RNG.

1.4 Random variate generation

So far we have learned how to obtain a reliable stream of numbers uniformly distributed in $[0, 1)$. Given such a stream, we can compute values for arbitrary distributions. Unfortunately, there is no general approach that works for every distribution. Some common methods are the `inverse transform` and the `convolution`.

1.4.1 Inverse transform

Suppose we need a sample x from a random variable with a given CDF $F(x)$ that is continuous and strictly increasing, and let F^{-1} be the inverse function of F. The method of inverse transform is:

1. generate $u \sim U[0, 1)$;
2. return $x = F^{-1}(u)$.

The idea is simple: the codomain of a CDF is $[0, 1]$, so a uniform number in $[0, 1)$ can be mapped back to a value of the domain. We can use this method only if we are able to obtain F^{-1}.

Example 1.4.
Suppose we want samples from an exponentially distributed random variable with mean $1/\lambda$. Its PDF and CDF are

$$f(x) = \lambda e^{-\lambda x}, \qquad F(x) = 1 - e^{-\lambda x}, \qquad x \geq 0.$$

We take u uniformly distributed in $[0, 1)$ and solve $u = 1 - e^{-\lambda x}$ for x:

$$x = -\frac{1}{\lambda} \ln(1 - u).$$